VAULTX PROTOCOL • DECENTRALIZED FINANCE VAULT

The Complete Plain-English Project Guide & Workflow

An intuitive, end-to-end walkthrough of architecture, smart contract security, and key codes

Project Name:	**VAULTX** — Self-Custody Finance Vault
Live Network:	Ethereum Sepolia Testnet (Chain ID: 11155111)
Deployed Contract:	**0xc105C3d4006d546D9eB92b72FEC2A1C18BcF027e**
Etherscan Link:	https://sepolia.etherscan.io/address/0xc105C3d4006d546D9eB92b72FEC2A1C18BcF027e
GitHub Code:	https://github.com/Hassaan1805/vaultX

1. The Big Picture: What is VAULTX in Plain English?

Imagine a modern **digital Swiss bank vault**. In a traditional bank, you give your money to a teller. The bank can freeze your account, lend your money out, or go bankrupt (as happened with FTX or Silicon Valley Bank).

VAULTX is completely different: It is a program that runs on the Ethereum blockchain (a *Smart Contract*). When you deposit your Ether (ETH) into VAULTX, **no human, no company, and no government can ever touch or freeze it**. The only person on Earth who can unlock your funds is **you**, using the private key in your MetaMask wallet. That is called **Self-Custody**.

■ The Real-World Analogy

Think of VAULTX like a wall of transparent, unpickable glass lockers. Each locker has an unbreakable digital lock that only your MetaMask key can open. Anyone can see how much money is inside (transparency), but only the rightful owner can unlock their individual locker (self-custody).

Why We Built VAULTX (The 3 Core Pillars):

1. **100% Self-Custody:** 'Your Assets. Your Control.' No accounts, passwords, or emails. You authenticate with your wallet.
2. **Transparent Accounting:** Every deposit and withdrawal prints an immutable digital receipt on Ethereum (Events).
3. **Unhackable Logic:** The smart contract follows strict security rules (CEI and ReentrancyGuard) to prevent theft.

2. The Whole Workflow (Step-by-Step)

Here is the complete journey of how funds and data move from the user's screen all the way into the Ethereum blockchain:

```
[Step 1: Connect]  User clicks 'Connect' -> MetaMask signs handshake -> Wallet address detected
       |
[Step 2: Deposit]  User enters 0.01 ETH -> Gas estimated -> MetaMask approval prompt
       |
[Step 3: Escrow]   Contract receives ETH -> Updates balances[user] -> Emits 'Deposit' event
       |
[Step 4: Custody]  ETH is securely held in contract on Sepolia; only user can withdraw
       |
[Step 5: Withdraw] User enters amount -> Contract checks balance -> Deducts state -> Sends ETH
       |
[Step 6: Activity] Live event listener catches tx -> Updates table with Sepolia Etherscan link
```

Detailed Walkthrough of Each Step:

Step 1: Connecting MetaMask (The Cryptographic Handshake)

There are no username/password sign-up forms in Web3. When you click **'Connect MetaMask'**, the dApp requests your public address (`0xBdA3A...775B9`). The dApp reads your current network (Ethereum Sepolia, Chain ID `11155111`). If you are accidentally on Mainnet or Polygon, the app displays a single-click button to automatically switch networks.

Step 2: Depositing ETH into the Vault

1. You type an amount (e.g., `0.01 ETH`) or click quick percentages (`25%`, `50%`, `MAX`).
2. **The Gas Buffer Safety Feature:** In normal dApps, clicking 'MAX' uses 100% of your ETH, causing MetaMask to error because you have no money left to pay the network gas fee. VAULTX automatically holds back **0.003 ETH** as a gas cushion so your deposit always succeeds!
3. You click **'DEPOSIT ETH TO VAULT'**. MetaMask pops up showing the transaction details.
4. You click **Confirm**. MetaMask sends the transaction to Ethereum Sepolia validators.
5. Once included in a block, the smart contract updates your balance, confetti bursts on your screen, and your vault balance updates live!

Step 3: Secure Escrow (Where the Money Lives)

The deposited ETH is held directly inside the FinanceVault smart contract address on the Sepolia blockchain. The contract stores your balance in an internal ledger: balances[your_address] = 0.01 ETH. There are no owner backdoors. The contract code has no function that allows the creator to touch your money.

Step 4: Withdrawing Your ETH

Whenever you want your money back, you enter the amount and click **'WITHDRAW ETH'**. The smart contract performs the **Checks-Effects-Interactions (CEI)** security pattern: it checks that you have enough funds, **deducts your balance first**, and only then transfers the native ETH back to your MetaMask wallet. This makes it impossible for hackers to steal funds via reentrancy.

Step 5: Live Activity Tracking

Every deposit and withdrawal emits an on-chain event. The frontend listens for these events in real-time, instantly adding each transaction to the **Transaction Activity** table with its block number, timestamp, and a direct link to view it on Sepolia Etherscan.

3. Key Smart Contract Code (`FinanceVault.sol`)

Here are the most critical lines of Solidity code in the smart contract, explained simply:

Code Snippet 1: How the Contract Remembers Balances (State Storage)

```
// Maps each user's wallet address to their current vault balance in wei
mapping(address => uint256) public balances;

// Keeps track of the total lifetime ETH ever deposited by each user
mapping(address => uint256) public userDeposited;

// Keeps track of the total lifetime ETH ever withdrawn by each user
mapping(address => uint256) public userWithdrawn;

// Global counter for total ETH ever deposited across all users
uint256 public totalVaultDeposited;
```

In Simple Words: A mapping in Solidity is like a digital spreadsheet. The first column is the wallet address (e.g. `0xBdA3A...`), and the second column is how much ETH that address owns. Because each person has their own row, User A can never see or modify User B's balance.

Code Snippet 2: The Deposit Function (`deposit`)

```
function deposit() external payable nonReentrant {
    // 1. Make sure the user didn't send 0 ETH
    if (msg.value == 0) {
        revert ZeroAmount();
    }

    uint256 amount = msg.value;

    // 2. Add the deposited ETH to the user's vault balance
    balances[msg.sender] += amount;
    userDeposited[msg.sender] += amount;
    totalVaultDeposited += amount;

    // 3. Emit an on-chain event so the frontend knows immediately
    emit Deposit(msg.sender, amount, block.timestamp);
}
```

In Simple Words: The keyword payable means this function can accept real cryptocurrency. msg.value is the exact amount of ETH sent in the transaction. msg.sender is the wallet address making the deposit. The contract adds the amount to balances[msg.sender] and emits a receipt (`Deposit` event).

4. The Withdrawal Code & Security Architecture

Withdrawals are the most dangerous part of any Web3 vault. If written incorrectly, a hacker can drain all user funds in seconds (like the famous $60M DAO hack). Here is how VAULTX makes withdrawals 100% safe:

Code Snippet 3: The Safe Withdrawal Function (`withdraw`)

```
function withdraw(uint256 amount) external nonReentrant {
    // STEP 1: CHECKS - Validate the request
    if (amount == 0) revert ZeroAmount();
    uint256 currentBalance = balances[msg.sender];
    if (amount > currentBalance) {
        revert InsufficientBalance(amount, currentBalance);
    }

    // STEP 2: EFFECTS - Update our ledger BEFORE sending money
    balances[msg.sender] = currentBalance - amount;
    userWithdrawn[msg.sender] += amount;
    totalVaultWithdrawn += amount;

    emit Withdrawal(msg.sender, amount, block.timestamp);

    // STEP 3: INTERACTIONS - Safely transfer the ETH back to user
    (bool success, ) = msg.sender.call{value: amount}("");
    if (!success) {
        revert TransferFailed();
    }
}
```

■■ What is Checks-Effects-Interactions (CEI)?

Notice that in Step 2, we subtract the user's balance BEFORE sending the ETH in Step 3! Why? If a hacker creates a malicious contract that calls withdraw() over and over in a loop (a Reentrancy attack), their balance is ALREADY zero on the second loop, so the attack immediately fails! Combined with OpenZeppelin's nonReentrant lock, the vault cannot be drained.

Code Snippet 4: The Fallback Function (`receive`)

```
// If someone sends ETH directly to the vault address without clicking deposit,
// this function catches it and automatically credits their account!
receive() external payable {
    if (msg.value == 0) revert ZeroAmount();
    balances[msg.sender] += msg.value;
    userDeposited[msg.sender] += msg.value;
    totalVaultDeposited += msg.value;
    emit Deposit(msg.sender, msg.value, block.timestamp);
}
```

5. Key Frontend Web3 Codes (React + ethers.js v6)

The frontend has two smart innovations that make the user experience silky smooth:

Code Snippet 5: Dual-Source Balance Fetching (`useWallet.ts`)

```
// 1. Ask MetaMask what balance it sees
const hexBal = await window.ethereum.request({
    method: 'eth_getBalance',
    params: [account, 'latest'],
});
if (hexBal) activeRaw = BigInt(hexBal);

// 2. Also check Sepolia public RPC directly!
const sepProvider = new ethers.JsonRpcProvider('https://ethereum-sepolia-rpc.publicnode.com');
const sepoliaRaw = await sepProvider.getBalance(account);

// If MetaMask is accidentally on Mainnet (0 ETH), the app detects your 0.0500 SepoliaETH
// and gives you a 1-click button to switch to Sepolia!
```

In Simple Words: In MetaMask's new interface, you might see 0.0500 SepoliaETH in your token list, but MetaMask is still connected to Ethereum Mainnet (where you have 0 ETH). Our code queries the Sepolia network directly so the app always knows your real testnet balance, preventing confusion!

Code Snippet 6: The 0.003 ETH Gas Buffer (`calculatePercentageAmount`)

```
export function calculatePercentageAmount(rawBalance, percentage, isDeposit) {
    if (rawBalance <= 0n) return '0';
    if (isDeposit && percentage === 100) {
        // Reserve 0.003 ETH so MetaMask doesn't reject for 'insufficient gas'
        const gasBuffer = ethers.parseEther('0.003');
        if (rawBalance <= gasBuffer) return '0';
        return ethers.formatEther(rawBalance - gasBuffer);
    }
    return ethers.formatEther((rawBalance * BigInt(percentage)) / 100n);
}
```

In Simple Words: In Ethereum, sending a transaction costs a small gas fee. If you have 0.0500 ETH and click 'MAX', trying to deposit all 0.0500 ETH will fail because you have 0 ETH left for gas. This function reserves 0.003 ETH for the gas fee, letting you deposit the maximum possible amount seamlessly.

6. Security Testing & Viva / Exam Questions

This section prepares you for project presentations, viva questions, and technical evaluations:

Automated Test Suite Summary (12/12 Tests Passing):

Category	What was tested	Result
1. Initial State	Vault starts with 0 TVL and 0 user balances.	**PASSED**
2. Deposits	Single and multiple deposits update user and global stats correctly.	**PASSED**
3. Zero Check	Depositing or withdrawing 0 ETH reverts with ZeroAmount().	**PASSED**
4. Withdrawals	Partial and full withdrawals transfer the exact native Ether.	**PASSED**
5. Overdraft	Withdrawing more than deposited balance reverts immediately.	**PASSED**
6. Isolation	User A cannot withdraw User B's funds even if contract has balance.	**PASSED**
7. Reentrancy	Malicious attack contract attempting recursive drain is blocked.	**PASSED**

Top 4 Viva Questions & Exact Answers:

Q1: Why did you use Ethereum Sepolia instead of Mainnet?

Answer: Sepolia is the official Ethereum Proof-of-Stake testnet. It mimics Ethereum Mainnet exactly (EVM, gas mechanics, consensus), but uses testnet ETH with zero financial cost, making it safe for educational development and live demonstrations.

Q2: What is a Reentrancy attack and how did you prevent it?

Answer: A reentrancy attack occurs when an external contract calls back into the withdraw function before the first execution updates the user's balance, draining the vault. We prevent this using two layers: the **Checks-Effects-Interactions (CEI)** pattern and OpenZeppelin's **ReentrancyGuard** (nonReentrant).

Q3: What happens if the website frontend crashes? Can users still get their money?

Answer: Yes, 100%! The money is held by the smart contract on the Ethereum blockchain, not by the website server. Any user can open Sepolia Etherscan, connect MetaMask, and call withdraw() directly on the smart contract to get their funds.

Q4: Does the contract owner have a secret backdoor to withdraw user funds?

Answer: No. There is intentionally no onlyOwner withdrawal function, pause lever, or admin role. The contract is completely self-custodial and trustless.

7. How to Run the Project (Cheat Sheet)

Here are the exact terminal commands to install, test, and run VAULTX on any machine:

```
# 1. Clone repository from GitHub
git clone https://github.com/Hassaan1805/vaultX.git
cd vaultX

# 2. Install dependencies for smart contracts and frontend
npm --prefix contracts install
npm --prefix frontend install

# 3. Run smart contract unit tests (Verifies all 12 security tests)
npm --prefix contracts test

# 4. Start the frontend development server
npm run dev
# -> Open your browser at http://localhost:3000

# 5. (Optional) Re-deploy to Sepolia with your own wallet
# Put your PRIVATE_KEY in .env, then run:
npm --prefix contracts run deploy:sepolia
```

8. Summary & Conclusion

VAULTX proves that a decentralized application can combine high-security cryptographic guarantees with an ultra-clean, intuitive user experience. By enforcing **Checks-Effects-Interactions**, employing **OpenZeppelin's ReentrancyGuard**, building an **editorial fintech UI** with Tailwind CSS, and implementing **dual-source balance detection**, the project bridges the gap between complex blockchain mechanics and accessible, real-world finance. It stands as a complete, working computer engineering final-year blockchain project.